

RELATORIO TECNICO

Visualizador 3D de Poligonos

Trabalho academico de Computacao Grafica: leitura de modelos OBJ/MTL, montagem de malha renderizavel, renderizacao em canvas e transformacoes 3D em tempo real.

GRUPO

Rafael Reis, Gabriel Drews, Arthur Schirmer e Felipe Santos

REPOSITORIO

Aula-08---Criar-um-Visualizador-3D-de-Poligonos

AMBIENTE

HTML, CSS, JavaScript ES Modules e Node.js para servidor/testes

IA UTILIZADA

ChatGPT 5.5 e minimax-m2.5

1. Visao geral e escopo

O sistema desenvolvido e um visualizador interativo de modelos 3D carregados a partir de arquivos `.obj` e `.mtl`. A aplicacao roda diretamente no navegador, com uma pagina HTML que permite carregar arquivos locais ou escolher quatro modelos prontos usados na apresentacao: cubo, piramide, octaedro e cilindro octogonal.

O projeto foi organizado para refletir as quatro etapas do trabalho: parser, montagem da geometria, renderizacao e transformacoes 3D. Essa separacao evita que a leitura de arquivo, a estrutura da malha e o desenho em tela fiquem misturados no mesmo modulo.

ENTRADA

OBJ com vertices, UVs, normais, faces, `mtllib` e `usemtl`

SAIDA VISUAL

Canvas 2D com modo solido, wireframe ou solido com arestas

TRANSFORMACOES

Escala, rotacao e translacao por matriz de modelo

VALIDACAO

Testes automatizados com `node:test`

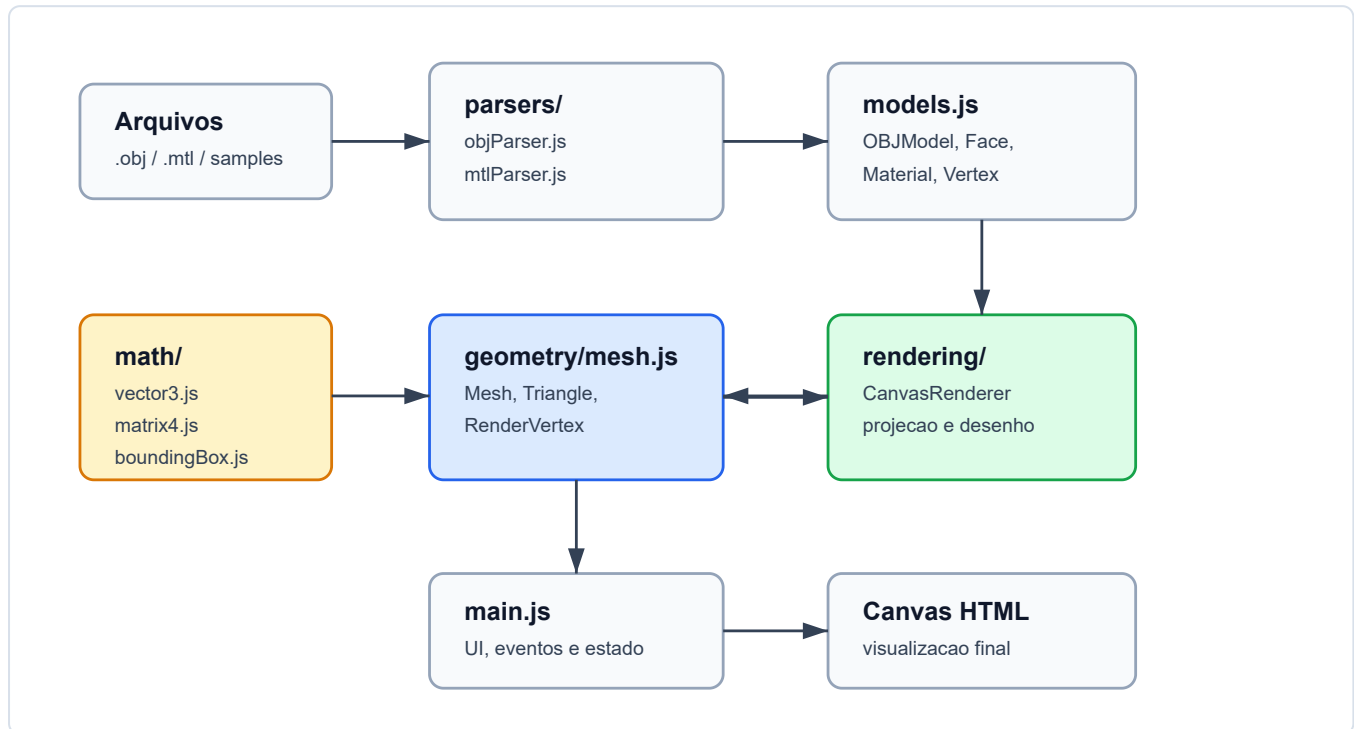
2. Biblioteca escolhida e justificativa

A biblioteca principal escolhida para a visualizacao foi a API nativa `CanvasRenderingContext2D` do navegador. A decisao foi manter a renderizacao sem bibliotecas externas como Three.js ou Babylon.js, porque o objetivo academico era explicitar as etapas de computacao grafica: parser, triangulacao, normalizacao da geometria, projecao, culling, iluminacao simples e transformacoes por matrizes.

O Node.js foi usado apenas como apoio de desenvolvimento: ele fornece um servidor estatico simples em `tools/server.js` e executa a suite de testes. Essa escolha reduz dependencias, facilita a execucao em sala e deixa o comportamento do visualizador concentrado no codigo do projeto.

3. Arquitetura do codigo

A arquitetura segue um fluxo de dados simples. O arquivo OBJ é parseado para um `OBJModel`, o modelo é convertido para uma `Mesh` renderizável e, em seguida, o renderizador transforma os vértices para espaço de câmera, projeta para 2D e desenha no canvas.



Responsabilidades principais

- **parsers/**: leitura linha a linha do OBJ/MTL e conversao para estruturas internas.
- **geometry/mesh.js**: triangulacao, normalizacao, centro, escala e normais.
- **math/**: vetores, produto vetorial, bounding box e matrizes 4x4.
- **rendering/renderer.js**: culling, projecao, iluminacao simples e desenho no canvas.
- **main.js**: carregamento, botoes, teclado, mouse e atualizacao dos indicadores da tela.

4. Deciso es tecnicas

Parser OBJ/MTL

O parser interpreta vertices (`v`), coordenadas UV (`vt`), normais (`vn`), faces (`f`), referencias `mtllib` e troca de material por `usemtl`. Os indices do OBJ sao convertidos para indices zero-based, simplificando o acesso aos arrays em JavaScript.

Montagem da malha

Faces com quatro ou mais vertices sao trianguladas por fan triangulation. A face `[0, 1, 2, 3]`, por exemplo, gera os triangulos `[0, 1, 2]` e `[0, 2, 3]`. O material da face original e preservado em cada triangulo.

Renderizacao e interacao

O renderizador trabalha com um pipeline simples: aplica matriz de modelo, converte para uma camera isometrica, remove faces traseiras pela normal em espaco de camera, calcula uma intensidade de luz direcional e desenha os triangulos. A profundidade media de cada triangulo e usada para ordenar o desenho.

A matriz de modelo e composta a partir de translacao, rotacoes X/Y/Z e escala uniforme. Os comandos de teclado escolhem a ferramenta ativa, e os valores atuais aparecem na interface para facilitar a demonstracao no video.

Normalizacao

A geometria e centralizada pela bounding box e escalada para caber em um intervalo normalizado. Isso permite carregar modelos com tamanhos diferentes sem perder o enquadramento inicial.

Normais

Quando o arquivo nao possui normais, o sistema calcula uma normal por face usando o produto vetorial $(v_1 - v_0) \times (v_2 - v_0)$ e normaliza o resultado. Normais existentes no OBJ sao preservadas e tambem normalizadas.

Recurso	Implementacao
Wireframe	Desenho das arestas visiveis de cada triangulo.
Solido	Preenchimento por triangulo com cor <code>kd</code> do MTL.
Projecao	Alternancia entre isometrica e perspectiva.
Transformacoes	Matrizes 4x4 aplicadas antes da camera e da projecao.

5. Dificuldades e solucoes adotadas

Dificuldade	Solucao adotada
OBJ permite faces com mais de tres vertices.	Conversao para triangulos por fan triangulation antes da renderizacao.
Modelos podem vir deslocados ou em escalas diferentes.	Calculo de bounding box, centro geometrico e escala normalizada.
Alguns arquivos podem nao possuir normais.	Geracao de normais por face com produto vetorial e normalizacao.
Visualizar solido sem WebGL exige ordenar faces e descartar costas.	Backface culling por normal em camera e ordenacao por profundidade media.
Demonstrar quatro modelos diferentes sem depender de busca externa.	Inclusao de cubo, piramide, octaedro e cilindro como samples locais.

6. Testes e validacao

A suite de testes usa `node:test` e valida parser, topologia, montagem de malha, renderizacao e os modelos de apresentacao. Os testes verificam leitura de vertices, UVs, normais e materiais; resolucao de indices negativos; triangulacao; centralizacao; escala normalizada; geracao de normais; projecao em perspectiva; e carregamento dos quatro samples.

Modelo	Vertices	Faces	Triangulos
Cubo	8	6	12
Piramide	5	5	6
Octaedro	6	8	8
Cilindro octogonal	16	10	28

7. Conclusao

O visualizador atende ao objetivo do trabalho ao expor o fluxo completo de um modelo 3D simples: leitura, organizacao da malha, renderizacao e transformacao interativa. A escolha por Canvas 2D e JavaScript puro favoreceu a compreensao das operacoes matematicas e manteve o projeto portavel para apresentacao em sala.

Ferramentas de IA utilizadas: ChatGPT 5.5 e minimax-m2.5.